@Query

Sunday, 15 June 2025 15:53

JPQL with @Query Annotation

- When you need more control or want to express complex queries, you can use the @Query annotation.
- JPQL (Java Persistence Query Language) is similar to SQL but works on the entity model rather than database tables.

How It Works:

- You annotate a repository method with @Query and provide a JPQL string.
- **Parameters can be passed using positional parameters (?1, ?2, ...) or named parameters (:paramName).**

1. **Custom JPQL Queries**

   Example:

```
public interface UserRepository extends JpaRepository<User, Long> {

  // Using named parameters
  @Query("SELECT u FROM User u WHERE u.email = :email")
  User findByEmail(@Param("email") String email);

  // Using positional parameters
  @Query("SELECT u FROM User u WHERE u.age > ?1")
  List<User> findUsersOlderThan(Integer age);

  // Complex query with join (assuming User has a relationship with Role)
  @Query("SELECT u FROM User u JOIN u.roles r WHERE r.name = :roleName")
  List<User> findUsersByRole(@Param("roleName") String roleName);
}
```

2. **Using Native SQL Queries**
   - vendor-specific SQL features or when the JPQL is not powerful enough, you can define a native SQL query by setting the nativeQuery attribute to true.
   - **Example:**

```
@Query(value = "SELECT * FROM users WHERE status = ?1", nativeQuery = true)
List<User> findUsersByStatus(String status);
```

3. **Supporting Update and Delete Operations**

- **Purpose:** The @Query annotation isn't limited to select queries. It can also be used for update and delete operations. When you do this, the method must be annotated with @Modifying to indicate that it modifies the database state.

Example (Update):

```
@Modifying
@Query("UPDATE User u SET u.status = :status WHERE u.id = :id")
int updateUserStatus(@Param("id") Long id, @Param("status") String status);
```

Example (Delete):

```
@Modifying
@Query("DELETE FROM User u WHERE u.lastLogin < :expirationDate")
int deleteInactiveUsers(@Param("expirationDate") LocalDate expirationDate);
```

Advantages:

- **Flexibility:** You can write almost any query you need, including complex joins, subqueries, and grouping.
- **Clarity:** For someone familiar with JPQL, the query's intent is explicit and clear.

Limitations:

- **Verbosity:** Writing JPQL can be more verbose compared to derived queries.
- **Maintenance:** Changes to the entity model require corresponding updates in JPQL expressions.

@Modifying :- having two fields **clearAutomatically** and **flushAutomatically**

Used on repository methods for DML operations (UPDATE, DELETE).

- **clearAutomatically=true:** Detaches entities from the persistence context after the query, preventing stale data. (**Recommended**)
- **flushAutomatically=true:** Flushes pending changes from the persistence context to the database before the query executes, ensuring visibility of prior changes. (Often **recommended**)